Azure Databricks – Delta Table Hands-on Exercises

Scenario: Courier Parcel Management

Participants will create Delta tables, perform CRUD operations, use MERGE/UPSERT, explore transaction history, perform Time Travel, and optimize tables.

Step 1 – Create the Delta table

Run this in an Azure Databricks notebook.

```
%sql
CREATE OR REPLACE TABLE courier_parcels_delta (
    parcel_id INT,
    customer_name STRING,
    destination_city STRING,
    service_type STRING,
    weight_kg DOUBLE,
    shipping_fee INT,
    delivery_status STRING
)
USING DELTA;
```

Step 2 – Insert sample data

```
%sql
INSERT INTO courier_parcels_delta VALUES
(101, 'Arjun', 'Hyderabad', 'Express', 2.5, 500, 'Delivered'),
(102, 'Meera', 'Mumbai', 'Standard', 4.0, 350, 'In Transit'),
(103, 'Karan', 'Delhi', 'Express', 1.5, 450, 'Pending'),
(104, 'Sneha', 'Pune', 'Standard', 3.0, 300, 'Delivered'),
(105, 'Rohit', 'Chennai', 'Express', 5.0, 800, 'In Transit'),
(106, 'Divya', 'Hyderabad', 'Standard', 2.0, 250, 'Cancelled'),
(107, 'Nikhil', 'Bengaluru', 'Express', 6.0, 950, 'Delivered'),
(108, 'Pooja', 'Mumbai', 'Standard', 1.0, 150, 'Pending'),
```

```
(109, 'Vikram', 'Delhi', 'Express', 3.5, 650, 'In Transit'),
(110, 'Anjali', 'Pune', 'Standard', 4.5, 400, 'Cancelled'),
(111, 'Farhan', 'Chennai', 'Express', 2.0, 550, 'Delivered'),
(112, 'Kavya', 'Hyderabad', 'Standard', 5.5, 600, 'Pending'),
(113, 'Manoj', 'Bengaluru', 'Express', 1.5, 400, 'In Transit'),
(114, 'Riya', 'Delhi', 'Standard', 3.0, 300, 'Delivered'),
(115, 'Tarun', 'Mumbai', 'Express', 4.0, 700, 'Pending');
```

Student exercises

Part A — Creating Delta tables

1. Display all records from `courier_parcels_delta`.
2. Create another Delta table named `courier_parcels_copy` using CTAS (CREATE TABLE AS SELECT).
3. Create a PySpark DataFrame containing three new parcel records and save it as a Delta table named `courier_parcels_pyspark`.
4. Verify that all three tables use the Delta format.

Part B — INSERT, UPDATE and DELETE

5. Insert a new parcel with ID 116, customer Sameer, destination Jaipur, Express service, weight 2.5 kg, shipping fee ₹550, and Pending status.
6. Update the delivery status of parcel 103 to `Delivered`.
7. Increase the shipping fee of all Express parcels by ₹100.
8. Change the destination city of parcel 108 to Hyderabad.
9. Delete all parcels with `Cancelled` status.
10. Display the remaining records ordered by shipping fee in descending order.

Part C — MERGE and UPSERT (SCD Type 1)

Use the following incoming data for the next exercises.

```
%sql
CREATE OR REPLACE TEMP VIEW incoming_parcels AS
SELECT *
FROM VALUES
    (102, 'Meera', 'Mumbai', 'Express', 4.0, 650, 'Delivered'),
    (105, 'Rohit', 'Chennai', 'Express', 5.0, 900, 'Delivered'),
    (117, 'Ayesha', 'Kolkata', 'Standard', 2.0, 300, 'Pending'),
```

```
    (118, 'Rahul', 'Jaipur', 'Express', 3.0, 600, 'In Transit')
AS incoming(
    parcel_id,
    customer_name,
    destination_city,
    service_type,
    weight_kg,
    shipping_fee,
    delivery_status
);
```

11. Display all records from `incoming_parcels`.
12. Identify which incoming parcel IDs already exist in the target Delta table.
13. Identify which incoming parcel IDs are new.
14. Perform a `MERGE` operation that updates all columns for existing parcels and inserts new parcels.
15. Verify that parcels 102 and 105 were updated.
16. Verify that parcels 117 and 118 were inserted.

Part D — Delta table internals and Time Travel

17. Execute `DESCRIBE DETAIL` and identify the table's format, storage location, number of files, and size.
18. Execute `DESCRIBE HISTORY` and identify the operations performed on the table.
19. Find the table version immediately before the MERGE operation.
20. Use `VERSION AS OF` to retrieve the records before the MERGE.
21. Compare parcel 102 in the previous version with its current record.

Part E — OPTIMIZE

22. Execute `OPTIMIZE` on `courier_parcels_delta`.
23. Run `DESCRIBE DETAIL` again and compare the number of files before and after optimization.
24. Execute `OPTIMIZE courier_parcels_delta ZORDER BY (destination_city)` if the table does not use liquid clustering.
25. Display the transaction history again and identify the optimization operations.

Note: Because this is a small dataset, `OPTIMIZE` may not significantly change the file count or query performance.